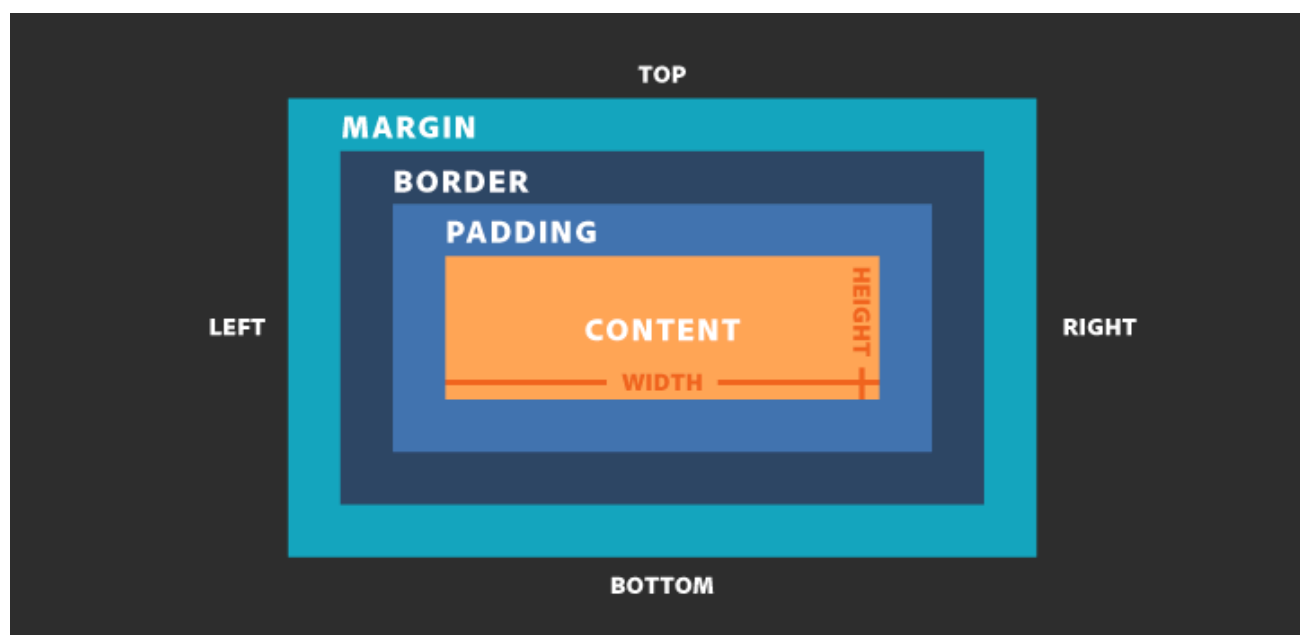


CSS Box Model (Modelo de Cajas)

Introducción al Box Model

La W3C define lo que se denomina box model (modelo de caja), que no es más que una zona rectangular que rodea cada uno de los elementos de nuestra página web.



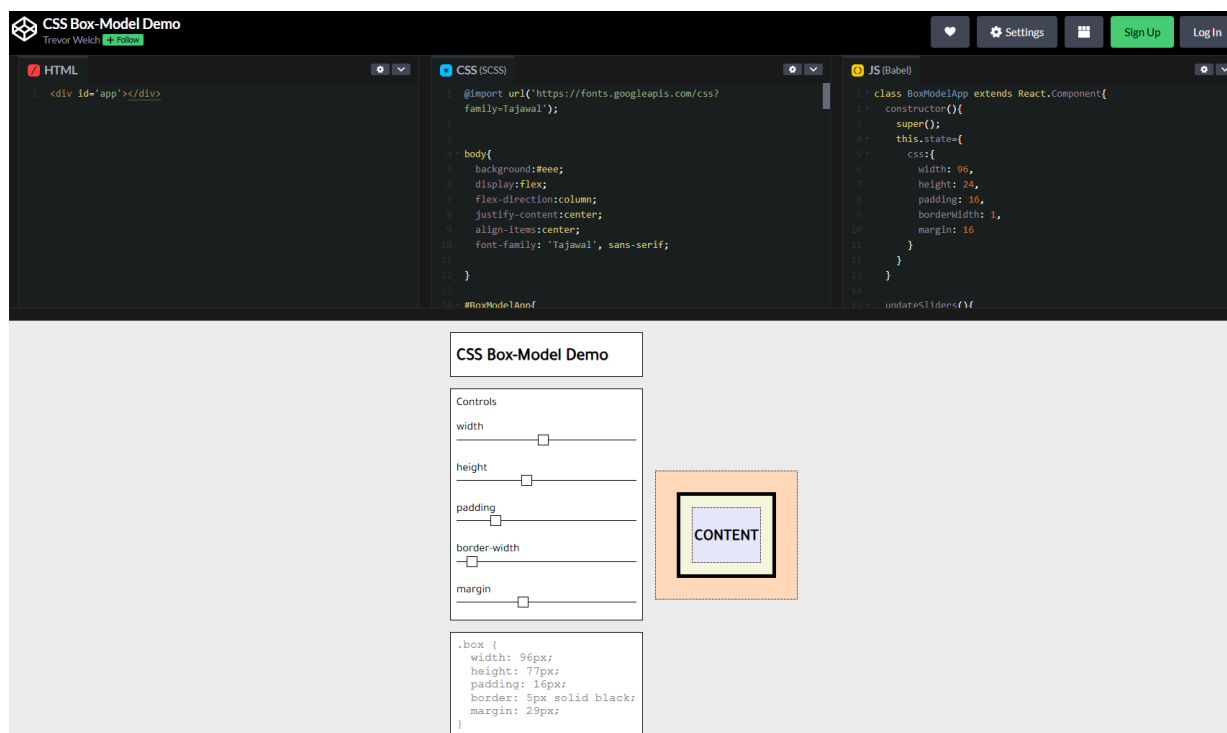
Cada etiqueta HTML aplica ese modelo y por lo tanto tiene:

- **Contenido.** Contenido que está en el interior de la etiqueta.
- **Margen interior.** Distancia desde el contenido al borde del elemento. Propiedad HTML `padding`.
- **Borde.** El borde del elemento. Propiedad HTML `border`.
- **Margen exterior.** Distancia desde el borde del elemento a los elementos adyacentes. Propiedad HTML `margin`.

Cada uno de esos elementos puede definirse mediante propiedades CSS.

EXPERIMENTA

Con la aplicación **Box Model Demo**, puedes experimentar de forma instantánea los cambios sobre las propiedades relacionadas con el modelo de cajas.



Propiedades de la Caja

Contenido

Para definir las dimensiones del contenido se utilizan las siguientes propiedades:

- `width`
- `height`

Los valores que pueden tomar son:

- `auto`
- Longitud en píxeles o equivalente

Ejemplo:

```
div {  
  width: 400px;  
  height: 100px;  
}
```



Tanto `width` como `height` especifican las dimensiones del contenido sin tener en cuenta borde y márgenes.

Margen interior (`padding`)

Con la propiedad `padding` podemos definir el ancho del margen interno.

```
p {  
  padding: 20px;  
}
```



Los valores que permiten son:

- `auto`
- Longitud
- Porcentaje

Variantes

Existen cuatro variantes de la propiedad:

```
p {  
  padding-top: 10px;  
  padding-bottom: 20px;  
  padding-right: 30px;  
  padding-left: 40px;  
}
```



Múltiples valores

La propiedad `padding` tiene diferentes comportamientos según los valores que le indiquemos:

- Con 1 valor: se aplica a los cuatro lados.
- Con 2 valores: el primero se aplica a superior e inferior y, el segundo, a los lados izquierdo y derecho.
- Con 3 valores: se aplica el primero, al superior; el segundo, a los laterales; y el tercero al inferior.
- Con 4 valores: se aplica a superior, derecho, inferior, izquierdo.

```
.e1 {  
  padding: 10px;  
  /* Todos los lados: 10px */  
}  
  
.e2 {  
  padding: 10px 20px;  
  /*  
    Arriba/abajo: 10px  
    Izquierda/derecha: 20px  
  */  
}  
  
.e3 {  
  padding: 10px 20px 30px;  
  /*  
    Arriba: 10px  
    Izquierda/derecha: 20px  
    Abajo: 30px  
  */  
}  
  
.e4 {  
  padding: 10px 20px 30px 40px;  
  /*  
    Arriba: 10px  
    Derecha: 20px  
    Abajo: 30px  
    Izquierda: 40px  
  */  
}
```



También es posible fijar el valor de cada una de los cuatro valores independientemente con la propiedad correspondiente.

Borde (**border**)

Propiedades que permiten aplicar estilos al borde de la caja son los mostrados a continuación.

border-color

Aplica un color de borde.

Los valores permitidos son:

- Color en alguna de las notaciones permitidas.
- `transparent`

Variantes (color) #

```
p {  
  border-top-color: red;  
  border-bottom-color: blue;  
  border-right-color: green;  
  border-left-color: yellow;  
}
```



Múltiples valores (color) #

La propiedad `border-color` tiene diferentes comportamientos según los valores que le indiquemos:

- Con 1 valor: se aplica a los cuatro lados.
- Con 2 valores: el primero se aplica a superior e inferior y, el segundo, a los lados izquierdo y derecho.
- Con 3 valores: se aplica el primero, al superior; el segundo, a los laterales; y el tercero al inferior.
- Con 4 valores: se aplica a superior, derecho, inferior, izquierdo.

```
p { border-color: red; }
p { border-color: red blue; }
p { border-color: red blue green; }
p { border-color: red blue green yellow; }
```



border-width #

Define el ancho del borde.

Los valores permitidos son:

- Una longitud.
- `thin`
- `medium`
- `thick`

Variantes (ancho) #

```
p {
  border-top-width: 1px;
  border-bottom-width: 2px;
  border-right-width: 3px;
  border-left-width: 4px;
}
```



Múltiples valores (ancho) #

La propiedad `border-width` tiene diferentes comportamientos según los valores que le indiquemos:

- Con 1 valor: se aplica a los cuatro lados.
- Con 2 valores: el primero se aplica a superior e inferior y, el segundo, a los lados izquierdo y derecho.
- Con 3 valores: se aplica el primero, al superior; el segundo, a los laterales; y el tercero al inferior.
- Con 4 valores: se aplica a superior, derecho, inferior, izquierdo.

```
p { border-width: 1px; }
p { border-width: 1px 2px; }
p { border-width: 1px 2px 3px; }
p { border-width: 1px 2px 3px 4px; }
```

border-style #

Define el estilo de la línea que delimita el borde.

Los valores permitidos son:

- solid
- dashed
- dotted
- double
- ridge
- groove
- inset
- outset
- hidden
- none

Variantes (estilo) #

```
p {
  border-top-style: solid;
  border-bottom-style: dashed;
  border-right-style: dotted;
  border-left-style: double;
}
```

Múltiples valores (estilo) #

La propiedad `border-style` tiene diferentes comportamientos según los valores que le indiquemos:

- Con 1 valor: se aplica a los cuatro lados.

- Con 2 valores: el primero se aplica a superior e inferior y, el segundo, a los lados izquierdo y derecho.
- Con 3 valores: se aplica el primero, al superior; el segundo, a los laterales; y el tercero al inferior.
- Con 4 valores: se aplica a superior, derecho, inferior, izquierdo.

```
p { border-style: solid; }  
p { border-style: solid dashed; }  
p { border-style: solid dashed dotted; }  
p { border-style: solid dashed dotted double; }
```



border-radius

Aplica un borde redondeado.

```
div {  
    border-radius: 50%;  
}
```



Los valores permitidos son:

- Una longitud.
- Un porcentaje.

Variantes (radio) #

```
div {  
    border-top-radius: 6px;  
    border-bottom-radius: 6px;  
    border-right-radius: 6px;  
    border-left-radius: 6px;  
}
```



Múltiples valores (radio) #

La propiedad `border-radius` tiene diferentes comportamientos según los valores que le indiquemos:

- Con 1 valor: se aplica a los cuatro lados.
- Con 2 valores: el primero se aplica a superior e inferior y, el segundo, a los lados izquierdo y derecho.
- Con 3 valores: se aplica el primero, al superior; el segundo, a los laterales; y el tercero al inferior.
- Con 4 valores: se aplica a superior, derecho, inferior, izquierdo.

```
p { border-radius: 1px; }  
p { border-radius: 1px 2px; }  
p { border-radius: 1px 2px 3px; }  
p { border-radius: 1px 2px 3px 4px; }
```



Margen exterior (**margin**)

Con la propiedad **margin** podemos definir el ancho del margen externo.

```
p {  
  margin: 20px;  
}
```



Los valores que permiten son:

- **auto**
- Longitud
- Porcentaje

Variantes (**margin**)

Existen cuatro variantes de la propiedad:

```
p {  
  margin-top: 10px;  
  margin-bottom: 20px;  
  margin-right: 30px;  
  margin-left: 40px;  
}
```



Múltiples valores (margin)

La propiedad `margin` tiene diferentes comportamientos según los valores que le indiquemos:

- Con 1 valor: se aplica a los cuatro lados.
- Con 2 valores: el primero se aplica a superior e inferior y, el segundo, a los lados izquierdo y derecho.
- Con 3 valores: se aplica el primero, al superior; el segundo, a los laterales; y el tercero al inferior.
- Con 4 valores: se aplica a superior, derecho, inferior, izquierdo.

```
.e1 {  
    margin: 10px;  
    /* Todos los lados: 10px */  
}  
  
.e2 {  
    margin: 10px 20px;  
    /*  
        Arriba/abajo: 10px  
        Izquierda/derecha: 20px  
    */  
}  
  
.e3 {  
    margin: 10px 20px 30px;  
    /*  
        Arriba: 10px  
        Izquierda/derecha: 20px  
        Abajo: 30px  
    */  
}  
  
.e4 {  
    margin: 10px 20px 30px 40px;  
    /*  
        Arriba: 10px  
        Derecha: 20px  
        Abajo: 30px  
        Izquierda: 40px  
    */  
}
```

```
*/  
}
```

También es posible fijar el valor de cada una de los cuatro valores independientemente con la propiedad correspondiente.

Unidades de Tamaño

Introducción

A la hora de especificar tamaños, CSS nos permite usar diferentes tipos de unidades.

Podemos realizar una clasificación global de todas las unidades en dos grupos:

- Unidades absolutas.
- Unidades relativas.

Unidades absolutas

Cualquier longitud expresada en una de estas unidades siempre se mostrará del mismo tamaño.

Las unidades absolutas disponible son las siguientes:

Unidad	Descripción
cm	Centímetros
mm	Milímetros
in	Pulgadas (inches)
px	Píxeles

pt	Puntos
pc	Picas

Normalmente, se utilizan unidades absolutas para la secciones y la estructura de la página web, porque se adaptan mejor a diferentes tamaños de pantalla.

Para las dimensiones de imágenes y vídeos se suelen utilizar píxeles.

Por ejemplo:

```
.video {  
  width: 1920px;  
  height: 1080px;  
}
```



i USO REAL

El píxel (px) es la única unidad absoluta que se utiliza habitualmente.

i TAMAÑO DE LOS PÍXELES

Se debe tener presente que el tamaño de un píxel varía entre un monitor y otro. Es decir, en 1 pulgada, podemos tener diferente cantidad de píxeles. La densidad de píxeles (cantidad de píxeles en un área determinada) varía en función de la resolución y la diagonal del monitor. La unidad habitual de densidad de píxeles es ppp (píxel por pulgada), es decir, representa la cantidad de píxeles en una pulgada.

Por ejemplo, podemos tener una resolución de 1920x1080 en un monitor de 27 pulgadas o en un teléfono móvil con una pantalla de 5 pulgadas. La cantidad de píxeles es la misma en ambos casos (la resolución es la misma), pero la densidad de píxeles varía, ya que en el teléfono móvil van a estar mucho más juntos.

Unidades relativas

Estas unidades define que el tamaño de un elemento dependa del tamaño de otro elemento.

Unidad	Descripción
em	Relativa al tamaño del tipo de letra por defecto.
ex	Relativa al valor de x-height de la fuente actual.
ch	Relativa al ancho del cero 0.
rem	Relativa al tamaño de letra del elemento raíz.
porcentajes (%)	Relativos a las dimensiones del elemento contenedor.

Ejemplo:

```
div {  
  width: 50%;  
  height: 4em;  
}
```



USO REAL

Las unidades más habituales son los porcentajes y la unidad `em`.

Ejemplos de unidades

Ejemplo 1 (porcentajes)

Supongamos el siguiente ejemplo:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <meta charset="UTF-8" />  
    <title>Porcentajes</title>
```



```

<style>
  #s1 {
    background-color: #7bed9f; /* verde */
    width: 100%;
  }
  #s2 {
    background-color: #70a1ff; /* azul */
    width: 50%;
  }
  #s3 {
    background-color: #eccc68; /* amarillo */
    width: 25%;
  }
</style>
</head>
<body>
  <section id="s1">Contenido sección 1</section>
  <section id="s2">Contenido sección 2</section>
  <section id="s3">Contenido sección 3</section>
</body>
</html>

```

En esta página se crean tres secciones:

- La primera tiene fondo verde y su anchura es el 100% del elemento contenedor. El elemento contenedor es `<body>`, que al ser un elemento de bloque, ocupa toda la anchura disponible.
- La segunda ocupa la mitad del elemento contenedor.
- La tercera ocupa la cuarta parte del elemento contenedor.

Si no fuese por el CSS, todas las secciones ocuparían toda la anchura.

En el navegador se visualizaría de la siguiente forma:



i PROBAR EN EL NAVEGADOR

Documento HTML

Ejemplo 2 (altura)

Supongamos el siguiente ejemplo:

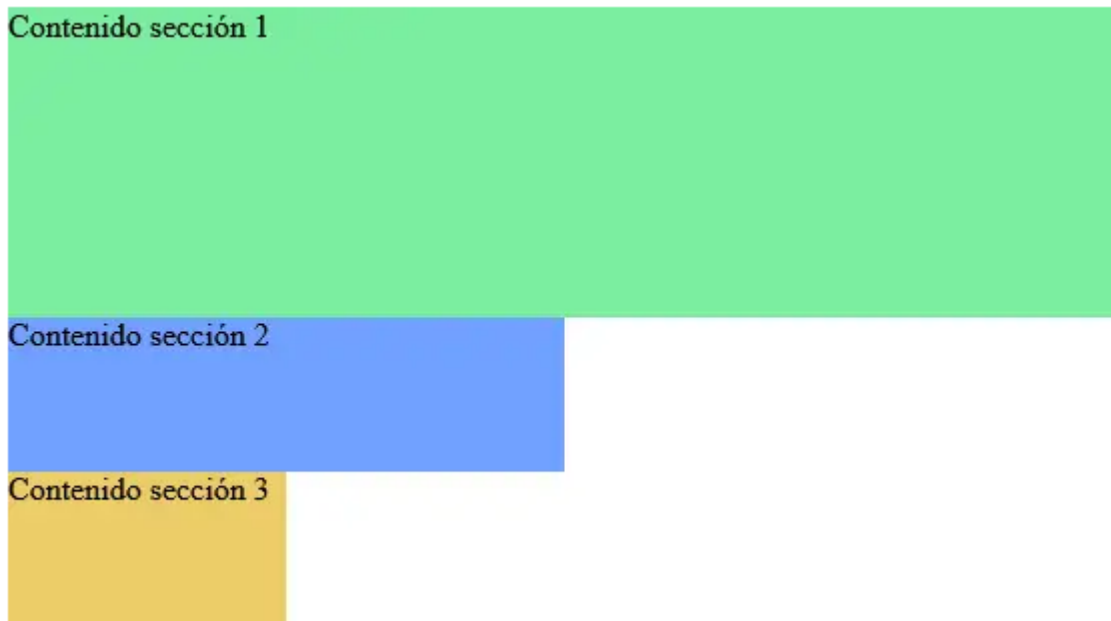
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Porcentajes</title>
    <style>
      html,
      body {
        height: 100%;
        margin: 0;
      }
      #s1 {
        background-color: #7bed9f; /* verde */
        width: 100%;
        height: 50%;
      }
      #s2 {
        background-color: #70a1ff; /* azul */
        width: 50%;
        height: 25%;
      }
      #s3 {
        background-color: #eccc68; /* amarillo */
        width: 25%;
        height: 25%;
      }
    </style>
  </head>
  <body>
    <section id="s1">Contenido sección 1</section>
    <section id="s2">Contenido sección 2</section>
    <section id="s3">Contenido sección 3</section>
  </body>
</html>
```

Con la altura (`height`) ocurre lo mismo que el caso de la anchura (`width`), pero hay que tener en cuenta que depende del contenido que haya en la página. A

no ser que se fije un valor para el elemento como en este ejemplo. Los elementos `<html>` y `<body>` tienen una altura fijada del 100%.

Las tres secciones tienen la altura y anchura expresadas en porcentajes relativos a los del elemento `<body>`.

En el navegador se visualizaría de la siguiente forma:



PROBAR EN EL NAVEGADOR

Documento HTML

Ejemplo 3 #

Supongamos el siguiente ejemplo:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Unidades em</title>
    <style>
      .grande {
        font-size: 2em;
      }
      .muy-grande {
```




```
        font-size: 4em;
    }
</style>
</head>
<body>
    <p>Normal</p>
    <p class="grande">Grande</p>
    <p class="muy-grande">Muy grande</p>
</body>
</html>
```

Para las fuentes es habitual utilizar la unidad `em`, que hace referencia al tamaño base de la fuente del documento HTML.

En este caso se está indicando que:

- Al contenido de la clase `grande` se le duplica el tamaño de la fuente.
- Al contenido de la clase `muy-grande` se le cuadriplica el tamaño de la fuente.

En el navegador se visualizaría de la siguiente forma:

Normal

Grande

Muy grande

 **PROBAR EN EL NAVEGADOR**

Documento HTML

Box-sizing (Tamaño de caja)

Cuando definimos un tamaño a un elemento y, a continuación, le añadimos un margen interior (`padding`), un borde (`border`) y un margen exterior (`margin`), su tamaño se va a ver modificado con respecto al tamaño original. El comportamiento sobre cómo afectan estas propiedades (`padding`, `border` y `margin`) al tamaño final de la caja, se pueden configurar con la propiedad `box-sizing`.

Por defecto, cuando se define un ancho o alto a un elemento (con las propiedades `width` o `height`, respectivamente), estamos definiendo el ancho o alto del contenido de un elemento.

Supongamos el siguiente ejemplo:



```
.hijo {  
  box-sizing: content-box;  
  width: 100%;  
}
```



En el ejemplo anterior, el hijo tiene un ancho del 100%, por lo tanto, ocupará el 100% del ancho del padre. El padre, al tener un ancho de 200 píxeles (sin contar el borde), el hijo tendrá un ancho de 200 píxeles.

PROBAR EN EL NAVEGADOR

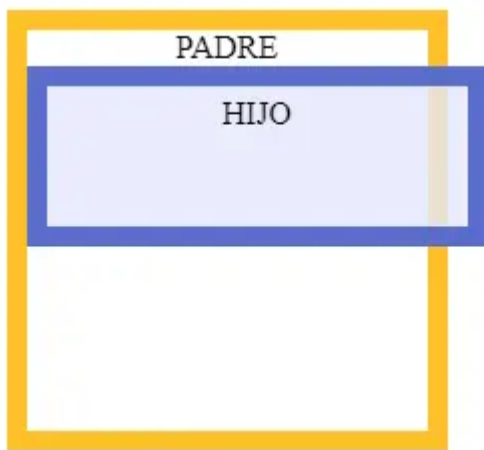
[Documento HTML](#)

Ahora, vamos a añadir un borde y margen interno al hijo:

```
.hijo {  
  box-sizing: content-box;  
  width: 100%;  
  border: solid #5B6DCD 10px;  
  padding: 5px;  
}
```



El resultado sería el siguiente:



Vemos que el child container desborda el parent container, ya que conserva el ancho que tenía anteriormente (el 100% de ancho es respecto al contenido) y añade 10 píxeles de borde y 5 píxeles de margen interno. Por lo tanto, el ancho total sería del 100% + 102 + 52, es decir, se aumentan 30 píxeles de ancho, de ahí ese desbordamiento.

i PROBAR EN EL NAVEGADOR

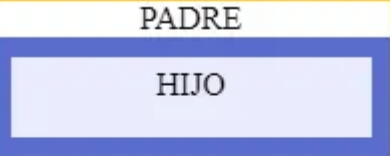
Documento HTML

Si queremos cambiar esta funcionalidad y que, una vez añadido el borde y el margen interior, encaje en el contenedor padre, debemos modificar el valor de la propiedad `box-sizing` a `border-box`:

```
.hijo {  
  box-sizing: border-box;  
  width: 100%;  
  border: solid #5B6DCD 10px;  
  padding: 5px;  
}
```



El resultado sería el siguiente:



Documento HTML

Ejemplo 1

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Relleno y margen</title>
    <style>
      #relleno {
        background-color: #eccc68; /* amarillo */
        padding: 2em; /*se aplica a izquierda, derecha arriba y abajo*/
      }
      #rellenoIzq {
        background-color: #ff7f50; /* naranja */
        padding-left: 2em; /*se aplica a izquierda */
      }
      #rellenoMargen {
        background-color: #70a1ff; /* azul */
        margin-bottom: 2em; /*se aplica abajo*/
      }
    </style>
  </head>
  <body>
```

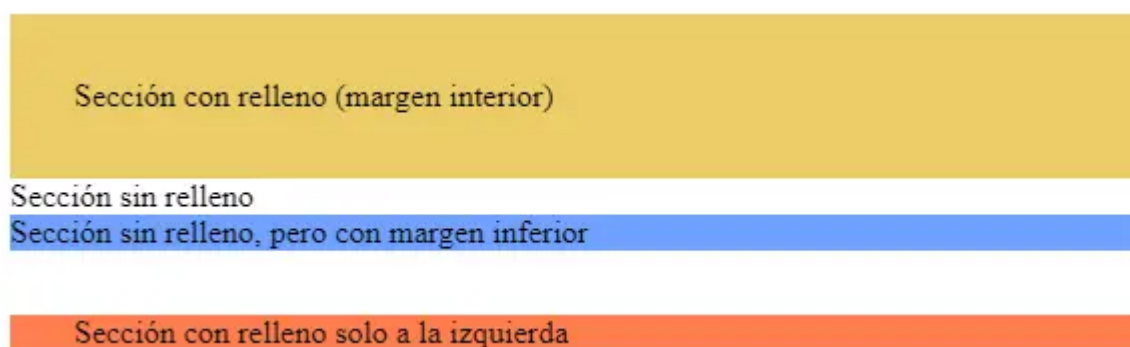
```

    </style>
</head>
<body>
    <section id="relleno">Sección con relleno (todos los lados)</section>
    <section id="rellenoIzq">Sección con relleno solo a la izquierda</section>
    <section id="rellenoMargen">Sección sin relleno, pero con margen inferior</section>
    <section id="rellenoIzq">Sección con relleno solo a la izquierda</section>
</body>
</html>

```

El espacio en blanco (que es el color de fondo predeterminado de `<body>`) entre las secciones azul y naranja se debe al margen inferior de la sección azul (`#rellenoIzq`).

En el navegador se visualizaría de la siguiente forma:



i PROBAR EN EL NAVEGADOR

[Documento HTML](#)

Ejemplo 2 #

Supongamos el siguiente documento HTML:

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Relleno y margen</title>
    <style>
      #borde1 {

```



```

        background-color: #ffa502; /* naranja */
        border-color: #2f3542; /* negro */
        border-style: solid;
        margin-bottom: 18px;
    }
    #borde2 {
        background-color: #ced6e0; /* gris */
        border-width: 4px;
        border-left-color: #ff4757; /* rojo */
        border-top-color: #ff4757; /* rojo */
        border-left-style: dashed;
        border-top-style: dashed;
    }
</style>
</head>
<body>
    <section id="borde1">Sección con borde1</section>
    <section id="borde2">Sección con borde2</section>
</body>
</html>

```

El espacio en blanco (que es el color de fondo predeterminado de `<body>`) entre las dos secciones se debe al margen inferior aplicado en la primera sección (`#borde1`).

En el navegador se visualizaría de la siguiente forma:



i PROBAR EN EL NAVEGADOR

Documento HTML

Posicionamiento

En los ejemplos vistos hasta ahora, el navegador representa los elementos en el orden en el que aparecen, atendiendo a si son elementos block o inline, entre

otras cosas.

Utilizando CSS, es posible modificar el posicionamiento por defecto de los elementos. Las propiedades implicadas son:

- `position`
- `float`

`float`

Con la propiedad `float`, los elementos «flotan» hacia la izquierda o derecha. Todos los elementos flotados se van situando uno junto a otro. Si no hay espacio disponible, se pasan a una nueva línea. Se adapta bastante bien a diferentes tamaños de pantalla.

Supongamos el siguiente ejemplo:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Etiquetas semánticas y float</title>
    <style>
      body {
        background-color: pink;
      }
      header {
        background-color: blue;
      }
      nav {
        background-color: red;
        width: 10%;
      }
      section {
        background-color: green;
        width: 90%;
      }
      footer {
        background-color: yellow;
      }
      nav,
```




```

    section {
        height: 300px;
        float: left;
    }
</style>
</head>
<body>
    <header>Encabezado</header>
    <nav>Vínculos</nav>
    <section>Contenido principal del página</section>
    <footer>Página creada por...</footer>
</body>
</html>

```

En este ejemplo, se utiliza la propiedad `float` para maquetar una página sencilla, junto a las etiquetas semánticas.

Los elementos `<nav>` y `<section>` están flotados a la izquierda y se reparten la anchura disponible.

Es necesario fijar la altura porque en la página apenas hay contenido.

En el navegador se visualizaría de la siguiente forma:



